

PRD-014 — Adaptive Recommendation Engine

Status: Ready for Implementation

Version: 1.0

Dependencies: PRD-012 — Diagnostic Assessment; PRD-013 — Question Content System

1. Purpose

The Adaptive Recommendation Engine generates personalized 21-question practice sets for each student.

The engine must:

- Maintain one persistent Ability Score for each SAT category.
- Initialize Ability Scores from the diagnostic.
- Update Ability Scores after every finalized answer.
- Prioritize categories needing more practice.
- Allocate questions across all seven categories.
- Select an appropriate difficulty for every question.
- Select exact questions from the published question bank.
- prevent duplicate questions and Question Families within a set.
- Resume unfinished sets without regenerating them.

The engine must remain understandable, configurable, and practical to recalibrate after V1 launches.

2. Supported Categories

Every student maintains independent adaptive state for exactly these seven categories:

1. Reading Comprehension
2. Grammar
3. Vocabulary
4. Algebra
5. Geometry & Trigonometry

6. Advanced Math
7. Problem Solving & Data Analysis

Performance in one category must not directly modify another category's Ability Score.

3. Core Configuration

All values in this section must be stored in one centralized adaptive-engine configuration module.

They must not be duplicated as hardcoded values throughout the codebase.

Configuration	V1 value
Practice-set size	21
Guaranteed questions per category	1
Additional priority-allocated questions	14
Maximum questions from one category	5
Minimum Ability Score	0.0
Maximum Ability Score	100.0
Maximum ability change per answer	5.0
Recent-performance window	5 category questions
Weakness priority weight	0.80
Recent-struggle priority weight	0.15
Focus-recency priority weight	0.05

The maximum of five questions per category is fixed for all students in V1 but must remain developer-configurable.

Configuration changes must apply only to newly generated sets. They must not modify an existing Active Practice Set.

4. Student Adaptive State

The engine must store one Category State per student per category.

Category State

studentId
category
ability
initialAbility
adaptiveQuestionsAnswered
consecutiveSetsWithoutExtraAllocation
lastAbilityUpdatedAt

Field behavior

ability

- Decimal from 0.0 through 100.0
- Initialized by the diagnostic
- Updated after every finalized adaptive-practice answer
- Never rounded during internal calculations

initialAbility

- Ability assigned after the diagnostic
- Must remain unchanged for analytics

adaptiveQuestionsAnswered

- Begins at 0 after the diagnostic
- Diagnostic questions do not count
- Increases once for every finalized adaptive-practice answer
- Used to determine the K-value

consecutiveSetsWithoutExtraAllocation

- Begins at 0
- Increases when a completed set contains exactly one question from the category
- Resets to 0 when a completed set contains two or more questions from the category
- Does not change until a set is completed

The engine must not maintain a confidence score.

5. Diagnostic Initialization

The diagnostic contains one Easy, one Medium, and one Hard question in each category.

The three results initialize that category's Ability Score using this table:

Easy	Medium	Hard	Initial ability
Incorrect	Incorrect	Incorrect	25.0
Correct	Incorrect	Incorrect	40.0
Incorrect	Correct	Incorrect	45.0
Correct	Correct	Incorrect	65.0
Incorrect	Incorrect	Correct	50.0
Correct	Incorrect	Correct	75.0
Incorrect	Correct	Correct	80.0
Correct	Correct	Correct	90.0

The table must remain configurable.

After diagnostic completion, the engine must create all seven Category State records before generating the first adaptive set.

Diagnostic answers initialize ability but must not:

- Count toward `adaptiveQuestionsAnswered`
- Enter the Recent Struggle calculation
- Receive normal Elo-style updates

6. Ability Score Updates

6.1 Question ratings

Every difficulty has one fixed V1 rating:

Difficulty	Rating
------------	--------

Easy	35.0
------	------

Medium	60.0
--------	------

Hard	85.0
------	------

Individual questions cannot have custom ratings in V1.

6.2 Expected probability

Before updating ability, calculate the probability that the student was expected to answer correctly:

$$E = \frac{1}{1 + 10^{(Q-A)/40}}$$

Where:

- (E) = expected probability of a correct answer
 - (Q) = question rating
 - (A) = category Ability Score immediately before the answer
-

6.3 K-value stages

The K-value is determined independently for each category.

Previously answered adaptive questions in category	K
0–9	10.0
10–29	8.0
30+	6.0

The count must be checked before processing the current answer.

Example:

- The tenth adaptive answer uses $K = 10$.
- The eleventh adaptive answer uses $K = 8$.

6.4 Update formula

Set:

```
[
R =
\begin{cases}
1 & \text{if correct} \\
0 & \text{if incorrect}
\end{cases}
]
```

Calculate the raw change:

```
[
\Delta_{\text{raw}} = K(R - E)
]
```

Apply the per-question cap:

```
[
\Delta = \operatorname{clamp}(\Delta_{\text{raw}}, -5, +5)
]
```

Calculate the new ability:

```
[
A_{\text{new}} = \operatorname{clamp}(A_{\text{old}} + \Delta, 0, 100)
]
```

Ability must be stored with decimal precision.

Required behavior

- Correct answers increase ability.
- Incorrect answers decrease ability.
- Unexpected outcomes cause larger changes.
- Expected outcomes cause smaller changes.
- One answer cannot change ability by more than five points.
- Ability cannot fall below 0 or rise above 100.
- The update must be applied exactly once per finalized answer.

7. Category Priority

Priority is recalculated only when generating a new practice set.

It must not alter an already generated set.

For each category, calculate:

1. Weakness Score
2. Recent Struggle Score
3. Focus Recency Score
4. Final Priority Score

7.1 Weakness Score

[
 $W = 100 - A$
]

Where (A) is the current Ability Score.

A lower ability produces a higher Weakness Score.

7.2 Recent Struggle Score

Use the five most recently finalized adaptive-practice answers in the category.

Diagnostic answers must not be included.

For each included answer:

- Use the expected probability stored or calculated immediately before that answer.
- Set ($R_i = 1$) for correct and ($R_i = 0$) for incorrect.

Calculate:

[

S_{raw}

$$100 \times \frac{\sum_{i=1}^n (E_i - R_i)}{n}$$

Then:

$$S = \operatorname{clamp}(S_{\text{raw}}, 0, 100)$$

Where (n) is between 1 and 5.

If no adaptive answers exist:

$$S = 0$$

This calculation measures whether the student performed worse than expected, not merely their raw accuracy.

A correct answer may offset recent unexpected mistakes.

7.3 Focus Recency Score

Focus Recency measures how many consecutive completed sets gave the category only its guaranteed one question.

Consecutive sets with no additional allocation	Focus score
0	0
1	25
2	50
3	75
4+	100

Equivalent formula:

$$[$$

$$F = \min(25N, 100)$$

$$]$$

Where (N) is `consecutiveSetsWithoutExtraAllocation`.

7.4 Final Priority Score

$$[$$

$$P = 0.80W + 0.15S + 0.05F$$

$$]$$

Where:

- (P) = Category Priority Score
- (W) = Weakness Score
- (S) = Recent Struggle Score
- (F) = Focus Recency Score

Priority Scores are internal and are not shown to students.

8. Category Allocation

Every set must contain exactly 21 questions.

Step 1: Guaranteed allocation

Allocate one question to each category.

7 categories × 1 question = 7 questions

Step 2: Additional allocation

Allocate the remaining 14 questions proportionally according to Category Priority Scores.

For each category:

$$[$$

$$I_i = 14 \times \frac{P_i}{\sum P}$$

$$]$$

Where (I_i) is the category's ideal number of additional questions.

Use the largest-remainder method:

1. Assign the floor of every ideal allocation.
2. Count the remaining unallocated slots.
3. Assign remaining slots to categories with the largest fractional remainders.
4. Break exact ties using the fixed category order defined in Section 2.
5. Enforce the maximum of four additional questions per category, producing five total questions including the guaranteed question.
6. If a category reaches the cap, redistribute excess slots among uncapped categories using the same process.

If all seven Priority Scores equal zero, distribute the fourteen additional slots evenly: two per category.

Required invariants

- Every category receives at least one question.
 - No category receives more than five questions.
 - Exactly fourteen additional slots are allocated.
 - The final category counts total exactly 21.
-

9. Difficulty Selection

After category counts are finalized, assign one difficulty independently to every slot.

Use the category's Ability Score at set-generation time.

Category ability	Eas y	Medium	Hard
0–29.999...	75%	25%	0%
30–44.999...	55%	45%	0%
45–59.999...	40%	50%	10%
60–74.999...	25%	60%	15%
75–89.999...	15%	55%	30%
90–100	5%	35%	60%

The percentages represent weighted random probability for every individual slot.

Example for ability 82:

15% Easy
55% Medium
30% Hard

If the category receives four questions, difficulty must be sampled four separate times.

The engine does not need to force each individual set to match the percentages exactly.

10. Practice Set Blueprint

After category and difficulty selection, create 21 Blueprint Slots.

Each slot contains:

position
plannedCategory
plannedDifficulty
resolvedCategory
resolvedDifficulty
questionId
questionFamilyId

Initially:

resolvedCategory = plannedCategory
resolvedDifficulty = plannedDifficulty

The resolved values may change only through content fallback.

The blueprint must be randomly ordered after all category and difficulty specifications are created.

Random ordering must not change:

- Category allocations
- Difficulty selections
- Total question count

The entire completed blueprint and all selected question IDs must be persisted before the set is shown to the student.

Once active, the blueprint must never be regenerated or modified because of later Ability Score changes.

11. Exact Question Selection

For each Blueprint Slot, retrieve published questions matching its resolved category and difficulty.

Exclude:

- Questions already selected in the current set
- Questions belonging to a Question Family already selected in the current set

Divide eligible questions into two tiers.

Tier 1

Exact questions the student has never previously answered.

This includes unseen versions from a Question Family the student has encountered before.

Tier 2

Exact questions the student has previously answered.

Selection rules

1. Randomly select from Tier 1 when Tier 1 is not empty.
2. Use Tier 2 only when Tier 1 is empty.
3. Within Tier 2, prefer the least recently answered exact question.
4. Randomly select among questions sharing the same oldest answer timestamp.
5. Prior exposure to a Question Family must not lower an unseen exact question's tier.

Repetition rules

Within one set:

- An Exact Question may appear only once.
- A Question Family may appear only once.

Across different sets:

- No Question Family cooldown exists.
 - No Exact Question cooldown exists.
 - The same family may appear in consecutive sets.
 - Previously answered questions may be reused indefinitely.
-

12. Content Availability Fallback

If no eligible question exists for a planned category and difficulty, use this order.

12.1 Same-category difficulty fallback

For a planned Easy slot:

1. Easy
2. Medium
3. Hard

For a planned Medium slot:

1. Medium
2. Easy
3. Hard

For a planned Hard slot:

1. Hard
2. Medium
3. Easy

At every difficulty level, apply the normal Tier 1 and Tier 2 selection rules.

12.2 Category reallocation

If no eligible question from the planned category can be selected:

1. Identify the highest-priority category below the five-question cap.
2. Sample a new difficulty using that category's Ability Score and difficulty distribution.
3. Attempt question selection in that category.
4. Continue through the next-highest-priority eligible categories if necessary.
5. Persist the original planned values and the final resolved values.

12.3 Generation failure

If the engine cannot construct a valid 21-question set:

- The set must not become active.
- The student must not receive an incomplete set.
- The failure must be logged.
- The system must report which category and difficulty pools prevented generation.

The engine must never use:

- An unpublished question
 - The same Exact Question twice
 - Two versions from the same family in one set
 - A question missing required PRD-013 publishing requirements
-

13. Practice Set Lifecycle

A student may have no more than one Active Practice Set.

Generation

When the student requests adaptive practice:

if an Active Practice Set exists:

 return the existing set

else:

 generate and persist a new 21-question set

Navigation

Students may:

- Move freely between questions
- Leave questions blank temporarily
- Change an unsubmitted answer
- Close the application and return later

The system must preserve:

- The set
- Question order

- Draft answers
- Finalized answers
- Current position
- Ability updates already applied

Answer finalization

An answer may remain a draft until submitted.

When an answer is finalized:

1. Evaluate the final submitted answer.
2. Mark it correct or incorrect.
3. Apply the Ability Score update immediately.
4. Increase **adaptiveQuestionsAnswered** for the category.
5. Store the expected probability, K-value, and ability change.
6. Prevent that answer from being updated a second time.

Only the final submitted answer affects the adaptive engine.

Intermediate answer selections must not affect ability.

Set submission with blanks

When the student attempts to complete a set containing blank questions:

1. Warn that unanswered questions remain.
2. Allow the student to return to them.
3. Allow the student to confirm submission anyway.
4. Treat every remaining blank question as incorrect.
5. Apply the standard incorrect-answer Ability Score update to each blank.
6. Mark all 21 questions finalized.

Completion

A set is complete only when all 21 questions are finalized.

Upon completion:

- Mark the set completed.
- Remove it as the student's Active Practice Set.
- Update all seven Focus Recency counters.
- Use the updated Ability Scores when the next set is generated.

A partially completed set must never be discarded or automatically replaced.

14. Required Persistence

The implementation must persist enough data to reproduce and audit adaptive behavior.

Category State

studentId
category
ability
initialAbility
adaptiveQuestionsAnswered
consecutiveSetsWithoutExtraAllocation

Practice Set

practiceSetId
studentId
status
engineVersion
randomSeed
createdAt
completedAt
currentPosition

Category Generation Snapshot

For each category in each set:

abilityAtGeneration
weaknessScore
recentStruggleScore
focusRecencyScore
priorityScore
guaranteedSlotCount
additionalSlotCount

Blueprint Slot

position
plannedCategory
plannedDifficulty

resolvedCategory
resolvedDifficulty
questionId
questionFamilyId

Finalized Attempt

studentId
practiceSetId
questionId
category
difficulty
answer
isBlank
isCorrect
expectedProbability
kValue
abilityBefore
rawAbilityChange
appliedAbilityChange
abilityAfter
finalizedAt
engineVersion

Finalizing an answer and updating ability must be atomic and idempotent.

A retried request must not apply the same Ability Score change twice.

15. Generation Pseudocode

```
function generatePracticeSet(student):  
  
    if student has Active Practice Set:  
        return existing Active Practice Set  
  
    categorySnapshots = []  
  
    for category in ALL_CATEGORIES:  
        state = loadCategoryState(student, category)  
  
        weakness = 100 - state.ability
```

```

struggle = calculateRecentStruggle(student, category, window=5)
focus = min(
    25 * state.consecutiveSetsWithoutExtraAllocation,
    100
)

priority =
    0.80 * weakness +
    0.15 * struggle +
    0.05 * focus

categorySnapshots.add({
    category,
    ability: state.ability,
    weakness,
    struggle,
    focus,
    priority
})

allocations = allocateCategories(
    snapshots=categorySnapshots,
    guaranteedPerCategory=1,
    additionalSlots=14,
    maximumPerCategory=5
)

slots = []

for allocation in allocations:
    repeat allocation.totalQuestions times:
        difficulty = weightedRandom(
            distributionFor(allocation.ability)
        )

        slots.add({
            plannedCategory: allocation.category,
            plannedDifficulty: difficulty
        })

randomlyShuffle(slots)

for slot in slots:
    question = selectQuestionWithFallback(

```

```
    student,  
    slot,  
    questionsAlreadySelected,  
    familiesAlreadySelected  
)
```

```
if question does not exist:  
    fail generation
```

```
slot.resolvedCategory = question.category  
slot.resolvedDifficulty = question.difficulty  
slot.questionId = question.id  
slot.questionFamilyId = question.familyId
```

```
persist snapshots, slots, questions, random seed, and engine version  
mark set Active
```

```
return set
```

16. Ability Update Pseudocode

```
function finalizeAnswer(attempt, finalAnswer):
```

```
    if attempt is already finalized:  
        return existing finalized result
```

```
    categoryState = loadCategoryState(  
        attempt.studentId,  
        attempt.category  
    )
```

```
    abilityBefore = categoryState.ability  
    questionRating = ratingFor(attempt.difficulty)
```

```
    expected =  
        1 /  
        (1 + 10 ^ ((questionRating - abilityBefore) / 40))
```

```
    result = finalAnswer is correct ? 1 : 0
```

```
    k = kValueFor(  
        categoryState.adaptiveQuestionsAnswered
```

)

```
rawChange = k * (result - expected)
appliedChange = clamp(rawChange, -5, 5)
```

```
abilityAfter = clamp(
  abilityBefore + appliedChange,
  0,
  100
)
```

atomically:

- finalize attempt
- store result and calculation values
- update categoryState.ability
- increment adaptiveQuestionsAnswered

17. Engine Invariants

The following must always remain true:

- Every student has exactly seven independent Category States after diagnostic completion.
- Ability Scores remain between 0.0 and 100.0.
- Ability calculations retain decimal precision.
- Every finalized adaptive answer changes ability exactly once.
- Every practice set contains exactly 21 questions.
- Every category appears at least once.
- No category appears more than five times.
- No Exact Question appears twice in one set.
- No Question Family appears twice in one set.
- Every selected question is published and valid under PRD-013.
- An Active Practice Set is never regenerated because ability or configuration changed.
- Priority is recalculated only when generating a new set.
- Students resume unfinished sets from their saved progress.
- Blank questions count as incorrect only after confirmed final submission.
- Previously encountered Question Families remain eligible in later sets.

18. Acceptance Criteria

PRD-014 is correctly implemented when all of the following pass:

1. Every diagnostic result combination produces the configured initial Ability Score.
 2. A correct answer increases category ability.
 3. An incorrect answer decreases category ability.
 4. Unexpected results create larger changes than expected results.
 5. No answer changes ability by more than five points.
 6. The correct K stage is selected independently for every category.
 7. Recent Struggle uses expected versus actual results from the latest five category attempts.
 8. Priority uses the 80% / 15% / 5% formula.
 9. Every generated set contains exactly 21 questions.
 10. Every category receives at least one question.
 11. No category receives more than five questions.
 12. Difficulty selection follows the configured weighted distributions.
 13. Unseen Exact Questions are preferred over previously answered Exact Questions.
 14. Previously seen Question Families are not penalized across sets.
 15. Two versions from the same family never appear in one set.
 16. An unfinished set resumes without regenerating its blueprint.
 17. Finalized answers immediately update ability.
 18. Draft answer changes do not update ability.
 19. Confirmed blank submissions are processed as incorrect.
 20. Retried finalization requests cannot apply duplicate ability updates.
 21. Content fallback preserves category before changing category whenever possible.
 22. The engine never returns a set containing fewer than 21 valid questions.
-

19. Out of Scope for V1

The following are not included:

- Student confidence scores
- Time-spent-based scoring
- Subtopic-level adaptation
- Individual question ratings
- Machine-learned difficulty calibration
- Family cooldowns across sets
- Immediate same-family remediation inside a set
- Mid-set blueprint regeneration
- Different adaptive configurations by student or school
- Separate scored and unscored practice questions
- Numerical-answer tolerance changes beyond PRD-013

End of PRD-014